

Cargoservice — Sprint1

ISS 2026 — VALERIA CAPACCI & GINEVRA PENTOLI

[GOAL](#) [ARCHITETTURA](#) [MODELLO QAK](#) [CLASSI JAVA](#) [IOPORT E WEBSOCKET](#) [PANNELLO WEB](#)
[TEST PLAN](#) [RIEPILOGO](#) [SPRINT2](#)

Goal

Realizzare il sottosistema che gestisce l'accettazione/rifiuto di una richiesta di carico, il timeout dei 30 secondi, e l'interazione reale con lo `IOPort`: pulsante, sensore, display e LED. `cargoservice` riceve `loadrequest`, interroga `Hold`, risponde con `loadaccepted` / `loadrejected` e transita da Idle a Engaged; se il timeout scade torna a Idle liberando lo slot. Lo `IOPort` è realizzato come un secondo QActor, con un pannello di controllo web collegato via WebSocket che ne simula l'hardware fisico.

Architettura del sottosistema

Il sottosistema di Sprint1 è composto da due QActor che condividono lo stesso `Context ctxcargoservice` (per semplicità, come già in Sprint0): `cargoservice`, l'orchestratore che applica la logica di accettazione/rifiuto e gestisce il timeout, e `ioport`, che rappresenta il dispositivo fisico all'ingresso della stiva. Ciascuno dei due incapsula il proprio stato interno con `withobj`: `cargoservice` usa `Hold` per tenere traccia degli slot, `ioport` usa `IOPortStatus` per il flag del sensore di presenza container.

Il modello qak

Di seguito il modello `sprint1.qak` completo. `loadrequest` porta il parametro `OCCUPIED`, letto dal sensore di `ioport`; le risposte `loadaccepted` / `loadrejected` portano anche lo stato corrente della hold (`HOLD`), da mostrare sul display. I dispatch `pushButton` e `setOccupied` arrivano dalla GUI verso `ioport` (simulano

rispettivamente la pressione del pulsante e il sensore fisico), mentre `blinkLed` viaggia da `cargoservice` a `ioport` ogni volta che il sistema entra o esce dallo stato *engaged*.

`ioport` nello stato `idle` attende uno dei tre dispatch in arrivo dalla GUI o da `cargoservice`. Alla pressione del pulsante (`onButtonPressed`) legge `status.isOccupied()` e apre una `request` verso `cargoservice`; ricevuta la risposta (`showAccepted` / `showRejected`) aggiorna il display tramite `wsAdapter`; `onSetOccupied` aggiorna il sensore interno e lo notifica alla GUI; `onBlinkLed` pilota il LED e, quando questo si spegne, aggiorna anche il display per segnalare che lo slot è stato liberato.

System sprint1

```
Request loadrequest : loadrequest(OCCUPIED) // fa
richiesta di carico, OCCUPIED controlla se l'ingresso è libero o
occupato da un container
Reply loadaccepted : loadaccepted(SLOT,HOLD) for loadrequest //
accetta il carico, assegna lo SLOT e mostra lo stato della HOLD
Reply loadrejected : loadrejected(REASON,HOLD) for loadrequest
// rifiuta il carico, dà il motivo (REASON) e mostra lo stato della
HOLD

Dispatch blinkLed : blinkLed(STATE,HOLD) // STATE = on |
off, inviato da cargoservice a ioport
Dispatch pushButton : pushButton(NONE) // simula la
pressione del pulsante fisico dello IOPort
Dispatch setOccupied : setOccupied(FLAG) // simula il
sensore
Dispatch updateDisplay : updateDisplay(STATE,HOLD,MSG)

Context ctxcargoservice ip [host="localhost" port=8050]

QActor cargoservice context ctxcargoservice
  withobj hold using "utils.cargoservice.Hold()" {

  [# var Slot = -1
  var Occupied = false #]

  State idle initial {
    println("$name | idle, in attesa di richieste...") color
magenta
  }
  Transition t0 whenRequest loadrequest -> handleRequest

  State handleRequest {
```

```

        onMsg(loadrequest : loadrequest(X)) {
            [# Occupied = payloadArg(0).toBoolean() #]
            [# var H = "" + hold.displayStatus() + "" #]
            if [# Occupied == true #] {
                replyTo loadrequest with loadrejected :
loadrejected(retrylater,$H)
            } else {
                [# Slot = hold.reserveFirstFree() #]
                if [# Slot != -1 #] {
                    [# var H2 = "" + hold.displayStatus() + "" #]
                    forward ioport -m blinkLed : blinkLed(on,$H2)
                    replyTo loadrequest with loadaccepted :
loadaccepted($Slot,$H2)
                } else {
                    replyTo loadrequest with loadrejected :
loadrejected(full,$H)
                }
            }
        }
    }

    Goto engaged if [# Slot != -1 && Occupied == false #] else idle

    State engaged {
        println("$name | engaged") color magenta
    }
    Transition t1 whenTime 30000 -> disengaged

    State disengaged {
        println("$name | timeout scaduto, libero lo slot e torno
idle") color magenta
        [# hold.releaseReserved() #]
        [# var H = "" + hold.displayStatus() + "" #]
        forward ioport -m blinkLed : blinkLed(off,$H)
        [# Slot = -1 #]
    }
    Goto idle
}

QActor ioport context ctxcargoservice
    withobj status using "utils.cargoservice.IOPortStatus()" {

        [# var WsAdapter = utils.cargoservice.IOPortWsAdapter(this, 7070)
#]

        State idle initial {
            println("$name | idle, in attesa della pressione del
pulsante...") color cyan
        }
    }

```

```
Transition t0
    whenMsg pushButton -> onPressed
    whenMsg setOccupied -> onSetOccupied
    whenMsg blinkLed -> onBlinkLed

State onPressed {
    onMsg(pushButton : pushButton(X)) {
        println("$name | pulsante premuto, invio richiesta di
carico") color cyan
        [# var Occ = status.isOccupied() #]
        request cargoservice -m loadrequest : loadrequest($Occ)
    }
}

Transition t1
    whenReply loadaccepted -> showAccepted
    whenReply loadrejected -> showRejected
    whenMsg blinkLed -> onBlinkLed

State showAccepted {
    onMsg(loadaccepted : loadaccepted(SLOT,HOLD)) {
        [# var S = payloadArg(0) #]
        [# var H = payloadArg(1) #]
        println("$name | display -> richiesta accettata, slot=$S")
color cyan
        [# WsAdapter.updateDisplay("ENGAGED", H, "Richiesta
accettata - slot " + S) #]
    }
}

Goto idle

State showRejected {
    onMsg(loadrejected : loadrejected(REASON,HOLD)) {
        [# var R = payloadArg(0) #]
        [# var H = payloadArg(1) #]
        println("$name | display -> richiesta rifiutata,
reason=$R") color cyan
        [# WsAdapter.updateDisplay("IDLE", H, "Richiesta
rifiutata: " + R) #]
    }
}

Goto idle

State onSetOccupied {
    onMsg(setOccupied : setOccupied(FLAG)) {
        [# var F = payloadArg(0) #]
        [# status.setOccupied(F.toBoolean()) #]
        println("$name | sensore IOPort forzato a occupied=$F")
color cyan
    }
```

```

        [# WsAdapter.notifySensor(F) #]
    }
}
Goto idle

State onBlinkLed {
    onMsg(blinkLed : blinkLed(STATE,HOLD)) {
        [# var S = payloadArg(0) #]
        [# var H = payloadArg(1) #]
        println("$name | led -> $S") color cyan
        [# WsAdapter.setLed(S) #]
        if [# S == "off" #] {
            [# WsAdapter.updateDisplay("IDLE", H, "Timeout
scaduto: slot liberato, torno idle") #]
        }
    }
}
Goto idle
}

```

Classi Java

Hold

Rispetto alla **Hold** di SprintO sono state necessarie due modifiche, entrambe legate a requisiti che in SprintO non erano ancora stati implementati:

- **reserveFirstFree()** ora restituisce **int** invece di **String**: a ogni slot è associato un numero da 1 a 4, e **-1** è il valore sentinella che sostituisce il precedente **null** (assenza di slot disponibile).
- È stato aggiunto **releaseReserved()**, necessario per R2: se il cliente non porta il container entro 30 secondi, lo slot prenotato deve tornare libero e il sistema deve passare a *disengaged*, altrimenti la hold si riempirebbe di prenotazioni mai confermate.

```

public class Hold {
    public static final int NUM_SLOTS = 4;          // slot1..slot4

    public enum SlotState { FREE, RESERVED, OCCUPIED }

    private SlotState[] slots = {
        SlotState.FREE,
    }
}

```

```
        SlotState.FREE,
        SlotState.FREE,
        SlotState.FREE
    };

    private boolean slot50occupied = false;

    public boolean isSlot50occupied() {
        return slot50occupied;
    }

    public void setSlot50occupied(boolean occupied) {
        slot50occupied = occupied;
    }

    public int reserveFirstFree() {
        for (int i = 0; i < NUM_SLOTS; i++) {
            if (slots[i] == SlotState.RESERVED) {
                return -1; // al più una prenotazione attiva alla
volta
            }
        }
        for (int i = 0; i < NUM_SLOTS; i++) {
            if (slots[i] == SlotState.FREE) {
                slots[i] = SlotState.RESERVED;
                return (i + 1);
            }
        }
        return -1; // piena
    }

    public void markReservedOccupied() {
        for (int i = 0; i < NUM_SLOTS; i++) {
            if (slots[i] == SlotState.RESERVED) {
                slots[i] = SlotState.OCCUPIED;
                return;
            }
        }
    }

    public void releaseReserved() {
        for (int i = 0; i < NUM_SLOTS; i++) {
            if (slots[i] == SlotState.RESERVED) {
                slots[i] = SlotState.FREE;
                return;
            }
        }
    }
}
```

```
public boolean isFull() {
    for (SlotState s : slots) {
        if (s != SlotState.OCCUPIED) {
            return false;
        }
    }
    return true;
}

public String displayStatus() {
    int occupied = 0;
    for (SlotState s : slots) {
        if (s == SlotState.OCCUPIED) {
            occupied++;
        }
    }
    return occupied + "/" + NUM_SLOTS + " occupied";
}
}
```

IOPortStatus

IOPortStatus è un oggetto di stato che l'attore **ioport** incapsula con **withobj**. Tiene traccia del flag **occupied** letto dal sensore di presenza container, ancora non implementato.

```
public class IOPortStatus {

    private boolean occupied = false;

    public boolean isOccupied() {
        return occupied;
    }

    public void setOccupied(boolean occupied) {
        this.occupied = occupied;
    }
}
```

IOPort: integrazione WebSocket

Lo `IOPort` è un pannello di controllo servito via browser e collegato all'attore tramite WebSocket. L'integrazione segue il seguente schema: un handler HTTP/WebSocket (nel nostro caso `IOPortWsAdapter`, basato su Javalin) riceve nel costruttore un riferimento all'attore a cui è associato, e lo usa per tradurre i messaggi in arrivo dalla pagina in dispatch qak da recapitare all'attore stesso.

```
package utils.cargoservice;

import java.util.Set;
import java.util.concurrent.CopyOnWriteArraySet;

import io.javalin.Javalin;
import io.javalin.http.staticfiles.Location;
import io.javalin.websocket.WsContext;
import it.unibo.kactor.ActorBasic;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import unibo.basicomm23.interfaces.IApplMessage;
import unibo.basicomm23.utils.CommUtils;

public class IOPortWsAdapter {

    private static final String SENDER = "ioportws";
    private static final String IOPORT_NAME = "ioport"; // nome
    dell'attore nel .qak, non generico di proposito

    private final ActorBasic ioport;
    private final Set<WsContext> clients = new CopyOnWriteArraySet<>
();

    public IOPortWsAdapter(ActorBasic ioport, int port) {
        this.ioport = ioport;

        var app = Javalin.create(config -> {
            config.staticFiles.add(staticFiles -> {
                staticFiles.directory = "/page";
                staticFiles.location = Location.CLASSPATH;
            });
        }).start(port);

        app.get("/", ctx -> ctx.redirect("/ioport_gui.html"));

        app.ws("/ws/ioport", ws -> {
            ws.onConnect(ctx -> {
                clients.add(ctx);
                CommUtils.outgreen("IOPortWsAdapter | client
```

```
connected: " + ctx.session.getRemoteAddress());
    });
    ws.onClose(ctx -> {
        clients.remove(ctx);
        CommUtils.outyellow("IOPortWsAdapter | client
disconnected");
    });
    ws.onMessage(ctx -> handleClientMessage(ctx.message()));
});

    CommUtils.outgreen("IOPortWsAdapter | started on port " + port
+ " (gui: http://localhost:" + port + "/ioport_gui.html)");
}

    private void handleClientMessage(String raw) {
        try {
            String[] dispatch = toDispatch(raw);
            sendToActor(dispatch[0], dispatch[1]);
        } catch (Exception e) {
            CommUtils.outred("IOPortWsAdapter | bad message from
client: " + raw + " (" + e.getMessage() + ")");
        }
    }

    private void sendToActor(String msgId, String content) {
        IApplMessage dispatch = CommUtils.buildDispatch(SENDER, msgId,
content, IOPORT_NAME);
        ioport.sendMsgToMyself(dispatch);
    }

    public void updateDisplay(String state, String hold, String msg) {
        broadcast(displayJson(state, hold, msg));
    }

    public void setLed(String onOff) {
        broadcast(ledJson(onOff));
    }

    public void notifySensor(String flag) {
        broadcast(sensorJson(flag));
    }

    private void broadcast(String json) {
        clients.forEach(ctx -> ctx.send(json));
    }

    static String[] toDispatch(String raw) throws Exception {
```

```
JSONObject json = (JSONObject) new JSONParser().parse(raw);
String type = (String) json.get("type");
if ("pushButton".equals(type)) {
    return new String[] { "pushButton", "pushButton(0)" };
} else if ("setOccupied".equals(type)) {
    boolean value = Boolean.TRUE.equals(json.get("value"));
    return new String[] { "setOccupied", "setOccupied(" +
value + ")" };
}
throw new IllegalArgumentException("unknown message type: " +
type);
}

static String displayJson(String state, String hold, String msg) {
    JSONObject json = new JSONObject();
    json.put("type", "display");
    json.put("state", state);
    json.put("hold", hold);
    json.put("msg", msg);
    return json.toJSONString();
}

static String ledJson(String onOff) {
    JSONObject json = new JSONObject();
    json.put("type", "led");
    json.put("on", "on".equals(onOff));
    return json.toJSONString();
}

static String sensorJson(String flag) {
    JSONObject json = new JSONObject();
    json.put("type", "sensor");
    json.put("occupied", "true".equals(flag));
    return json.toJSONString();
}
}
```

Pannello di controllo web

La GUI (`ioport_gui.html`) riproduce il dispositivo descritto nei requisiti: uno "schermo" con stato/hold/timer/messaggio, un LED, il pulsante `Carica` e un interruttore che simula il sensore occupato.

```
<!DOCTYPE html>
<html lang="it">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>IOPort – Cargoservice</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="device">
  <div class="plate">
    <span class="name">IOPort</span>
    <span class="tag">nodo cargoservice</span>
  </div>

  <div class="screen">
    <div class="row"><span>stato</span><span id="stateVal">IDLE</span>
  </div>
    <div class="row"><span>hold</span><span id="holdVal">0/4</span>
  </div>
    <div class="row"><span>timer</span><span id="timerVal">--</span>
  </div>
    <div class="msg" id="msgVal">In attesa di richieste...</div>
  </div>

  <div class="status-row">
    <div class="led" id="led"></div>
    <span class="label">LED – engaged</span>
  </div>

  <div class="button-wrap">
    <button class="pushbutton" id="pushBtn">Carica</button>
  </div>

  <div class="sensor">
    <span class="label">Sensore occupato
      <small>simula presenza container (test)</small>
    </span>
    <button class="switch" id="sensorSwitch" aria-pressed="false"
      aria-label="Simula sensore occupato"></button>
  </div>

  <p class="footnote" id="connStatus">connessione in corso...</p>
</div>

<script src="wscontrol.js"></script>
```

```
</body>  
</html>
```

Lo script `wscontrol.js` apre la connessione WebSocket, gestisce la riconnessione automatica in caso di caduta (`onclose` richiama `connect` dopo `RECONNECT_MS`), e traduce i messaggi JSON in aggiornamenti del DOM.

```
(function () {  
  const btn = document.getElementById('pushBtn');  
  const sensorSwitch = document.getElementById('sensorSwitch');  
  const led = document.getElementById('led');  
  const stateVal = document.getElementById('stateVal');  
  const holdVal = document.getElementById('holdVal');  
  const msgVal = document.getElementById('msgVal');  
  const connStatus = document.getElementById('connStatus');  
  const timerVal = document.getElementById('timerVal');  
  
  let sensorOccupied = false;  
  let socket = null;  
  const RECONNECT_MS = 2000;  
  
  const TIMER_SECONDS = 30;  
  let timerInterval = null;  
  
  function startTimer() {  
    let remaining = TIMER_SECONDS;  
    timerVal.textContent = remaining + ' s';  
    btn.disabled = true;  
  
    clearInterval(timerInterval);  
    timerInterval = setInterval(() => {  
      remaining -= 1;  
      if (remaining <= 0) {  
        clearInterval(timerInterval);  
        timerVal.textContent = '--';  
        if (socket && socket.readyState === WebSocket.OPEN) {  
          btn.disabled = false;  
        }  
        return;  
      }  
      timerVal.textContent = remaining + ' s';  
    }, 1000);  
  }  
  
  function setConnStatus(text, lost) {
```

```
    connStatus.textContent = text;
    connStatus.style.color = lost ? '#e88' : '';
  }

function setControlsEnabled(enabled) {
  btn.disabled = !enabled;
  sensorSwitch.disabled = !enabled;
}

function connect() {
  socket = new WebSocket('ws://' + window.location.host +
    '/ws/ioport');

  socket.onopen = () => {
    setConnStatus('connesso', false);
    setControlsEnabled(true);
  };

  socket.onmessage = (event) => {
    let msg;
    try {
      msg = JSON.parse(event.data);
    } catch (e) {
      console.warn('ioport_gui: messaggio non JSON ignorato:',
event.data);
      return;
    }
    if (msg.type === 'display') {
      stateVal.textContent = msg.state;
      holdVal.textContent = msg.hold;
      msgVal.textContent = msg.msg;
    } else if (msg.type === 'led') {
      led.classList.toggle('on', !!msg.on);
    } else if (msg.type === 'sensor') {
      sensorOccupied = !!msg.occupied;
      sensorSwitch.classList.toggle('occupied', sensorOccupied);
      sensorSwitch.setAttribute('aria-pressed',
String(sensorOccupied));
    }
  };

  socket.onclose = () => {
    setConnStatus('connessione persa, riprovo...', true);
    setControlsEnabled(false);
    setTimeout(connect, RECONNECT_MS);
  };

  socket.onerror = () => {
```

```

        socket.close();
    };
}

btn.addEventListener('click', () => {
    if (socket && socket.readyState === WebSocket.OPEN) {
        socket.send(JSON.stringify({ type: 'pushButton' }));
        startTimer();
    }
});

sensorSwitch.addEventListener('click', () => {
    if (!(socket && socket.readyState === WebSocket.OPEN)) return;
    sensorOccupied = !sensorOccupied;
    sensorSwitch.classList.toggle('occupied', sensorOccupied);
    sensorSwitch.setAttribute('aria-pressed', String(sensorOccupied));
    socket.send(JSON.stringify({ type: 'setOccupied', value:
sensorOccupied }));
});

setControlsEnabled(false);
connect();
})();

```

Riepilogo di avanzamento

Rispetto alle macro-entità individuate in Sprint0, questa tabella riassume cosa è stato effettivamente realizzato in Sprint1 e cosa resta da fare.

Componente	Stato	Note
<code>cargoservice</code>	Fatto	QActor completo: stati Idle/Engaged/Disengaged, logica di accettazione/rifiuto, timeout 30 s
<code>Hold</code>	Fatto	Slot 1–4 con FREE/RESERVED/OCCUPIED
<code>ioport</code> (attore)	Fatto	QActor, gestisce pulsante, sensore, LED e display
Pannello web / WebSocket	Fatto	<code>IOPortWsAdapter</code> + GUI HTML/CSS/JS, simula pulsante, sensore, display, LED
Sonar	Da fare	Attore <code>sonar</code> , firmware; stato <i>Out of service</i> da gestire

<code>cargorobot</code>	Da fare	Logica di orchestrazione dei movimenti
Marker (slot5)	Da fare	Stazione di marcatura temporanea
Multi-Context	Da fare	Tutti gli attori condividono ancora un unico <code>ctxcargoservice</code> ; necessaria separazione in nodi fisici distinti.

Definizione dello Sprint2

Lo Sprint1 ha chiuso il core business (accettazione/rifiuto, timeout, IOPort come attore reale collegato a una GUI). Lo Sprint2 dovrà estendere il sistema con i componenti rimasti fuori da Sprint1:

REQUISITI

- Introdurre l'attore `sonar` : emissione degli eventi `sonarcontainer` / `sonarfault` a partire dalle letture di distanza, e gestione dello stato *Out of service* ($D > D_{FREE}$ per almeno 3 s) sul display dello IOPort.
- Progettare e realizzare l'attore `cargorobot` , che orchestra il movimento del DDR appoggiandosi al servizio esterno `RobotSmart26` : raggiungere l'IOPort, prelevare il container, passare da `slot5` (marcatura) allo slot riservato, tornare in HOME.
- Modellare il `marker` : segnalazione (interna al nodo `cargoservice`) del completamento della marcatura in `slot5` , prima che il robot possa prelevare il container.
- Separare i Context per nodo fisico (`cargoservice` , `cargorobot` , `sonar` , `ioport`), oggi ancora unificati in `ctxcargoservice` per semplicità.



Valeria Capacci

valeria.capacci@studio.unibo.it



Ginevra Pentoli

ginevra.pentoli@studio.unibo.it

Git: github.com/valecapa/TemaFinaleISS2026